Joshua Dalton
IT 505

1. What is the conceptual (as opposed to syntactical) distinction between a function statement and a function expression (aka, function literal)?

Functional statements declare functions on their own while function expressions create functions that can be stored in a variable and/or passed down to another function.

2. How would you declare a simple function using a function statement? What is a function expression, and how does it differ in behavior from a function statement? How would you declare a simple function using a function expression involving the function keyword? How would you declare a simple function using a function expression involving the arrow notation?

A function statement would use the function keyword to declare a function. A function expression creates a function that can be stored in a variable. Function statements can be called before they are declared and function expressions can't.

Declaring using keyword:

```
const bark = function(): void {
   console.log("Woof");
};
```

Declaring using simple:

```
function bark(): void {
   console.log("Woof");
}
```

Declaring using arrow:

```
const bark= (): void => {
   console.log("Woof");
};
```

3. How are type annotations used to define the types of a function's parameters? How are they used to define the type of a function's return value?

Type annotations are used to specify what types of values a function's parameters can accept as well as what type of value is returned by the function.

4. How do you invoke a function?

Writing its name then followed by parentheses with arguments inside them.

5. Explain what a first-class function is. Does the language support them?

A first-class function is a function that can be treated like any other value. TS does support this.

Joshua Dalton
IT 505

6. What is a function type and how are function types defined? How are function types related to the concept of first-class functions? What role, if any, do the parameter names play in a function type's definition?

A function type defines what types of parameters a function accepts as well as what type of value it returns. They let functions be treated as values while still checking their type. Parameter names don't matter.

7. Explain what a higher-order function is. Does the language support them? How does this relate to the concept of first-class functions?

A higher-order function is a function that takes another function as an argument or returns a function. TS supports this because functions are treated as values (first-class).

8. How are functions represented internally (in terms of their JavaScript data type)? What are some implications of this, in particular as it relates to first-class functions?

Functions are objects in JS, but typeof would identify them as a "function". This lets them be stored in variables and passed to other functions like other values.

9. Is there any distinction made between a function and a method? If so, explain that distinction.

Function: Block of code that performs a task

Method: Function that belongs to an object

10. How do you return a value from a function? What happens if a function does not explicitly generate a return value?

The return keyword. It would return undefined if the function doesn't generate a return value.

11. What happens if you provide a TypeScript function with more arguments than it has parameters? What about fewer arguments than parameters? Can you intentionally write functions that accept a variable number of arguments? How does this behavior differ in JavaScript, as opposed to TypeScript?

TS normally requires the correct number of arguments when calling a function. Providing the wrong amount causes a type checking error. Rest parameters allow functions to take a variable number of arguments. JS allows the incorrect amount of arguments without the error.

12. What does the rest operator do? How do you write one? How might one be useful? What is meant by the term rest parameter?

It collects multiple arguments into an array. A rest parameter allows a function to accept any number of arguments, which is useful when the amount of inputs isn't known.

13. What does the spread operator do in the context of function arguments? What is meant

by the term spread argument?

The spread operator takes values from an array (or other iterable) and passes them as separate arguments to a function. A spread argument is an argument that has been expanded in this manner.

14. Can you specify default values for parameters that will be used when no matching argument is provided at the point of invocation? If so, how?

Yes. By using = when declaring them. The default value is used if an argument is not provided/undefined. Example:

```
function greet(name: string = "Guest"): void {
   console.log("Hello " + name);
}

greet("Josh"); //prints Hello Josh
greet(); //prints Hello Guest
```

15. Are anonymous functions (functions that do not have identifiers associated with them as part of their definition) allowed? If so, what syntax is used to create them?

Yes. They can be created using the function keyword or arrow notation. Example syntax:

```
const bark= function() {
   console.log("Bark");
};
```

16. What is a callback function (or just callback for short) and why are they useful? What's important to remember if you wish to pass the reference to a function rather than its return value?

A callback is a function passed to another function that will be used later. They are useful because they allow functions to perform different tasks. To pass the function itself instead of its return value you need to leave out the parentheses.

17. How can you sort the elements of an array into a particular order? What role, if any, does the idea of a callback function play in this process? What is meant by a stable sort, and does this mechanism qualify as stable?

The sort() method. A callback function can be used to control how the elements are sorted. A stable sort keeps elements with equal values in their original order. JS' sort() method is considered stable.

18. The Array object offers several different methods for finding elements within an array. List them and explain how they differ from one another.

- **indexOf() = returns the first index of a value.**

Joshua Dalton
IT 505

- **lastIndexOf() = returns the last index of a value.**
- **includes() = checks if a value exists in an array.**
- **find() = returns the first element that meets a condition.**
- **findIndex() = returns the index of the first element that meets a condition.**
- **findLast() = returns the last element that meets a condition.**
- **findLastIndex() = returns the index of the last element that meets a condition.**

19. Explain the behavior of the Array object's forEach() method.

It goes through every element in an array and runs a callback function for each one.

20. Compare and contrast the filter() and map() methods of the Array object. How does the flatMap() method relate? How do reduce() and reduceRight() round out the picture?

filter() method creates a new array containing elements that meet a condition, map() creates a new array by changing each element. flatMap() works like map(), but also "flattens" the result by one level. reduce() combines array elements into one result from left to right, while reduceRight() does the same from right to left.

21. What does it mean to chain methods? Can this be accomplished in TypeScript? If so, give an example and discuss the advantage(s) of this technique.

Method chaining means calling multiple methods one after another. TS does support this, and it is useful because it allows multiple operations to be performed without making extra variables. Example:

```
let numbers: number[] = [1, 2, 3, 4, 5]; //creates an array of numbers

let result = numbers
   .filter(n => n > 2) //keeps numbers greater than 2
   .map(n => n * 2); //multiplies each remaining number by 2

console.log(result); //prints [6, 8, 10]
```

22. Explain the functional programming paradigm. List some ways in which it is supported by TypeScript.

Functional programming is a way of writing code that focuses on using functions to complete tasks instead of constantly changing data. TS supports this through things like first-class functions, callbacks, higher-order functions, and array methods such as map() and filter().

23. What is a closure? Provide a simple example. What types of applications can you envision for this mechanism?

Joshua Dalton
IT 505

A closure is when a function remembers variables from where it was created, even after the outer function has finished. Closures are useful for keeping track of values as well as protecting data. Simple example:

```
function counter() {
   let count = 0;
   return () => ++count;
}

let add = counter();

console.log(add()); //prints 1
console.log(add()); //prints 2
```

24. When using Deno, is it possible to establish code that will execute once after a timer has elapsed? If so, how? Can the timer be cancelled before it expires? If so, how?

Yes. Deno uses setTimeout() to run code once after a certain amount of time. The timer can be cancelled before it finishes using clearTimeout().

25. When using Deno, is it possible to establish code that will execute repeatedly after a timer has elapsed, automatically resetting the timer after each execution? If so, how? Can the repetition be cancelled? If so, how?

Yes. It uses setInterval() to repeatedly run code after a certain amount of time. This repetition can be stopped using clearInterval().

26. What is meant by the term refactoring when applied to code?

Changing how code is written to make it cleaner and easier to understand without changing what the program does.

27. Describe some ways that the clean code philosophy could be meaningfully applied to the material you've discovered in this unit

The clean code philosophy can be applied meaningfully by using clear variable and function names, keeping functions simple, and avoiding repeated code. One big thing I've learned from this unit is that functions should focus on doing a specific task instead of trying to do multiple things at once.